Minimal Functors, Applicatives, and Monads in Haskell

1. Introduction

Walking through the exercise of constructing "minimal" instances is one of the best ways to deeply understand Functors, Applicatives, and Monads in Haskell. By stripping away domain-specific noise (like state management, I/O, or failure), we demystify a lot of features that initially look like magic. It reveals the underlying mechanics at play.

While the core concepts structured here are foundational to modern Haskell, this specific teaching narrative—starting with absolute minimalism to actually "prove" the forced hand of parametricity—is something usually only found scattered across different resources. We will synthesize foundational ideas found in Philip Wadler's *"Theorems for free!"* and Sandy Maguire's *"Thinking with Types"*.

In this exploration, our scope is specific: we are focusing entirely on Endofunctors operating within the category of Haskell types (from `Hask` to `Hask`).

> **Note on `Hask`**: Technically, `Hask` is not a strict mathematical category due to non-terminating programs (represented by `_|_` or "bottom"). For the purposes of reasoning about types, we generally ignore this, a practice validated by the famous paper "Fast and Loose Reasoning is Morally Correct".

The true protagonist of this journey is **Parametricity**. Due to parametric polymorphism (the inability to inspect types at runtime), the implementation of most functor, applicative, and monad instances for simple structures is mathematically forced to be unique. This provides immense "intellectual economy" for Haskell developers: operations like `bind`, `pure`, `fmap`, `apply` (`<*>`), and Kleisli composition (`>=>`) generally have exactly one possible correct implementation for simple structural types. The compiler practically writes the code for you.

(Note: There are rare counterexamples where multiple valid implementations might exist—for example, traversing a complex tree structure in different orders—but these are atypical for the minimal types we are exploring).

Chapter 1: The Foundations of Functors, Applicatives, and Monads

Section 1.1: What is a Functor?

In Haskell, a functor is represented as a type constructor that maps from one type to another type; it has the kind `* -> *`.

Before we look at Haskell, let's briefly touch on Category Theory. In Category Theory, a functor is a mapping between categories. A Haskell `Functor` is actually an *Endofunctor* because it maps from the category `Hask` back to the category `Hask`. However, Haskell functors are just a finite subset of all possible categorical functors. For example: 1. **Non-Endofunctor**: A functor mapping from a

completely different category (e.g., the category of Sets) to `Hask`. 2. **Non-parametric Functor**: A categorical functor that inspects types (e.g., mapping `Int` to `String` and `Bool` to `Double`). This is strictly forbidden in Haskell due to parametricity. 3. **Restricted Functor**: A generic categorical functor might only apply to a *subset* of objects (types). A Haskell `Functor` mathematically must unconditionally apply to *all* types.

If we look at valid and invalid candidates in Haskell, it purely comes down to type signatures (kinds): * `Maybe` is a valid functor candidate because it has kind `* -> *`. It needs one concrete type (like `Int`) to become a concrete type (`Maybe Int`). * `Int` is an invalid candidate. It already has kind `*`.

A brief taxonomy note: In Haskell, it helps to distinguish how we bundle these types. `type` just defines a synonym. `newtype` is a single-constructor wrapper with zero runtime overhead, heavily used for isolating functor behaviors. `data` is a full algebraic data type capable of multiple constructors.

Section 1.2: The Constraint of Parametricity

To understand why our implementations will be mathematically forced, we must understand parametricity. When we write a polymorphic function in Haskell, the function must be completely ignorant of the types going into it.

If we have a generic type `a` and need to produce a generic type `b`, we cannot inspect the value, switch on its type, or conjure a `b` out of thin air. This drastic restriction essentially forces our implementations to **preserve structure**. This concept is famously codified in Philip Wadler's paper "Theorems for free!", which proves that simply reading the type signature of a polymorphic function tells you almost everything about what the function physically *must* do.

Section 1.3: `fmap` (and `<$>`) and the Functor Laws

The core of the Haskell `Functor` typeclass is `fmap`:

```
fmap :: (a -> b) -> f a -> f b
```

Compare this to standard function application, the `($)` operator:

```
($)  :: (a -> b) ->   a ->   b
```

`fmap` (often used as the infix operator `<$>`) is literally just standard function application lifted into a context `f`.

To be a lawful Functor, instances must obey two mathematical laws: 1. **Identity Law**: `fmap id == id` 2. **Composition Law**: `fmap (f . g) == (fmap f) . (fmap g)`

Due to parametricity ("theorems for free"), verifying the Identity Law usually naturally guarantees the Composition Law. If you preserve the structure flawlessly for `id`, you will preserve it flawlessly for any composed function.

A Crucial Note on Enforcement: Haskell, the language compiler, does *not* enforce these mathematical laws. It is code; it only checks type signatures. It is entirely the developer's responsibility to ensure their instances are lawful. Fortunately, automated property-based testing libraries like

`QuickCheck` (often alongside `tasty`) provide an extremely easy and robust way to mathematically test and guarantee that your data structures fulfill these laws across thousands of generated inputs.

Section 1.4: The Applicative Functor

An `Applicative` is a Functor equipped with two additional powers: 1. `pure :: a -> f a`: The ability to bring a raw value into the functor context. 2. `(<*>) :: f (a -> b) -> f a -> f b`: The ability to apply a function that is *already within* the context to a value within the context.

Like Functors, Applicatives must obey a set of laws (Identity, Composition, Homomorphism, and Interchange) which ensure that sequencing these context-applications behaves predictably.

Section 1.5: The Monad

The `Monad` adds the ultimate power: the ability to sequence context-dependent computations where subsequent steps depend on the unwrapped values of prior steps.

What makes Monads intellectually elegant is that there are **three mathematically equivalent paths** to define them. Because they are algebraically equivalent, providing a valid implementation for *any single one* of them (along with your `fmap` and `pure`) allows the other two to be derived entirely for free. *(For a formal treatment of this equivalence, refer to Eugenio Moggi's foundational paper "Notions of computation and monads" or the Typeclassopedia. See the Annex below for the algebraic proof).*

These three paths are: 1. **Bind (`>>=`)**: `m a -> (a -> m b) -> m b` 2. **Join (`mu`)**: `m (m a) -> m a` (Flattening heavily nested contexts) 3. **Kleisli composition (`>=>`)**: `(a -> m b) -> (b -> m c) -> (a -> m c)`

Often, `join` (called `mu` in Category Theory) is considered the most foundational and natural way to describe a Monad's raw structure, while `bind` is heavily favored in Haskell for practical ergonomics.

Chapter 2: The Minimal Endofunctor (and its Applicative/Monad)

Let's begin the exercise. What is the smallest possible Functor we can build in Haskell?

Section 2.1: The Smallest Valid Candidate

We need a type constructor of kind `* -> *` that holds absolutely the minimum amount of data possible. The answer is **none**.

```
data MinF a = Val
```

(Note: `Val` is the data constructor. In Haskell, value constructors must always begin with an uppercase letter).

`MinF` maps any phantom type `a` to a constructor that contains zero term-level data. The type `a` exists only at compile time; at runtime, the box is completely empty.

Section 2.2: A Singular Functor Implementation

Our task is to write `fmap`.

```
fmap :: (a -> b) -> MinF a -> MinF b
```

If we try to write this abstractly, we might say `fmap f val = val`. Let's apply this to our data structure:

```
instance Functor MinF where
    fmap _ Val = Val
```

The "Why": Due to parametricity, there is exactly one possible implementation that compiles. This phenomenon is famously referred to by author Sandy Maguire as the type signature "forcing your hand." We are given a function `(a -> b)`. We have a `MinF a`, whose only possible value is `Val`. We must return a `MinF b`, whose only possible value is `Val`. We do not have an `a` to feed into the function. Therefore, the function *must* be ignored. Out of mathematical necessity, `fmap _ Val = Val` is the unique, indisputable implementation.

Section 2.3: Upgrading to Applicative

Let's look at `pure` and `<*>`:

```
pure :: a -> MinF a
(<*>) :: MinF (a -> b) -> MinF a -> MinF b
```

The implementations:

```
instance Applicative MinF where
    pure _ = Val
    Val <*> Val = Val
```

The "Why": Again, our hands are tied. `pure` gives us an `a` and demands a `MinF a`. We cannot store the `a`, so we throw it away and return `Val`. The apply operator `<*>` receives two `Val`s and must return a `Val`. There is no other mathematical choice.

Section 2.4: Upgrading to Monad

We have three equivalent paths to upgrade to a Monad. Let's look at `join` and `bind`.

Via `join` (`mu`):

```
join :: MinF (MinF a) -> MinF a
```

Visualizing `mu`: Walking through the value-level representation, the input type is "A `MinF` whose phantom type happens to be another `MinF`". But at the physical value level, an empty box inside an empty box is still exactly just `Val`.

```
join Val = Val
```

Via `bind` (`>>=`):

```
(>>=) :: MinF a -> (a -> MinF b) -> MinF b
```

We receive `Val`. The only thing we can return is `Val`. We have no `a` to pass to the function `(a -> MinF b)`, so we ignore it.

```haskell
instance Monad MinF where
    Val >>= _ = Val
```

Haskell Equivalents: This minimal behavior—a structure carrying a phantom type but no value—is extremely useful. In the standard Haskell library, this is identically represented by `Proxy` (carrying zero values) or `Const r` (carrying some other value `r`, but no `a`).

Section 2.5: Let the Compiler Do the Work (`deriving`)

Because there is mathematically exactly *one* valid implementation for `MinF` due to parametricity, we don't actually need to write any of this code ourselves. By utilizing GHC extensions like `DeriveFunctor`, we can simply write:

```haskell
data MinF a = Val deriving (Functor)
```

The compiler mechanically generates the correct code for us because its constraints are exactly the same as ours: its "hands are tied."

Chapter 3: The Second Minimal Endofunctor (Identity)

Section 3.1: The Next Smallest Candidate

If `MinF` is the minimal structure with *no* value, what is the minimal structure with exactly *one* value?

```haskell
data IdF a = IdVal a
```

Unlike `MinF`, `IdF` actually possesses the `a` at the term level. It is a completely transparent wrapper.

Section 3.2: A Singular Functor Implementation

```haskell
instance Functor IdF where
    fmap f (IdVal x) = IdVal (f x)
```

The "Why": The type signature demands we produce an `IdF b`. Due to parametricity, we cannot inspect the type or summon a `b` from the æther. The *only* mathematical way to obtain a `b` is to take the `x` (which is of type `a`) that we possess inside `IdVal`, and apply our given function `f :: (a -> b)` to it.

Section 3.3: Upgrading to Applicative

```haskell
instance Applicative IdF where
    pure x = IdVal x
    IdVal f <*> IdVal x = IdVal (f x)
```

To implement `pure`, we are given an `x` and must wrap it. To implement `<*>`, we unwrap the function `f`, unwrap the value `x`, physically apply them, and rewrap the result. Parametricity allows no alternative.

Section 3.4: Upgrading to Monad

Let's witness the three equivalent paths for `IdF`.

Via `join` (`mu`):

```haskell
join :: IdF (IdF a) -> IdF a
join (IdVal (IdVal x)) = IdVal x
```

The only structurally preserving way to flatten nested `IdVals` into a single `IdVal`.

Via `bind` (`>>=`):

```haskell
IdVal x >>= f = f x
```

We take the `x` out of the wrapper and pass it to `f` (which returns an already-wrapped `IdF b`).

Via `kleisli` (`>=>`):

```haskell
(f >=> g) x = f x >>= g   -- Expanding out: (f >=> g) x = let (IdVal y) = f x in
g y
```

Haskell Equivalents: In the standard Haskell library, this completely transparent wrapper is known exactly as the `Identity` functor/monad.

Chapter 4: Conclusion: The Tale of Two Minimals

These two minimal structures—`IdF` and `MinF`—perfectly illustrate how Haskell's type system dictates physical behavior at the value level.

- **With `MinF` (Proxy/Const ())**: You *do not have* an `a`. Because you have no `a` to feed to the function `(a -> b)`, parametricity **forces** you to completely ignore the function.
- **With `IdF` (Identity)**: You *have* an `a`. Because you must produce a `b`, and you have a function `(a -> b)`, parametricity **forces** you to apply the function to the value.

By starting from the absolute minimal examples, the "magic" of Functors and Monads evaporates. All that remains is the elegant, inescapable logic of types.

Chapter 5: The Algebra of Functors (Sums and Products)

So far we have looked at the two absolute minimal building blocks of polynomial functors: 1. **The Constant Functor (`Const r`)**: Represents a constant value independent of the generic type `a` (like c in algebra). A special case is `Const ()`, which is natively known as `Proxy`. It carries absolutely zero term-level data, which makes it mathematically equivalent to the number 1. 2. **The Identity Functor (`IdF` / `Identity`)**: Represents the parameter itself (like X in algebra).

Every other standard algebraic data type can be built by adding (Sum types) and multiplying (Product types) these two foundational blocks together!

Before we do that, does an empty wrapper like `Proxy` actually have practical use? Yes! `Proxy` is heavily used at the term level to guide type inference without incurring runtime costs (e.g., querying the size in `Storable a => Proxy a -> Int`).

Chapter 6: Functors out of Proxy and Identity

Let's look at what happens when we combine our minimal blocks using the simplest algebraic data type operations: Sums and Products.

Section 6.1: The Sum (Maybe)

If we take the **Sum** (represented by the `Either` type, or $+$ in algebra) of `Proxy` (which is isomorphic to 1) and `Identity` (X), we get:

```
Sum Proxy Identity a  ≅  Either (Proxy a) (Identity a)  ≅  Either () a  ≅  Maybe a
```

Mathematically: $1 + X$

By summing the functor of *Zero computational data* and the functor of *One computational data*, we derive `Maybe`. It is the Minimal Monad of Failure or Choice.

Section 6.2: The Product (Identity)

What if we take the **Product** (`(,)`, or $\times$ in algebra) of `Proxy` and `Identity`?

```
Product Proxy Identity a  ≅  (Proxy a, Identity a)  ≅  ((), a)  ≅  Identity a
```

Mathematically: $1 \times X = X$

By multiplying a functor carrying zero data (just `()`) with the identity functor, we haven't added any new information. A tuple of `((), a)` carries no more information than just `a`. Therefore, the product stays as exactly the `Identity` functor! This proves that `Proxy` acts identically to the number 1 in multiplication.

(Note: Adding an arbitrary constant `Const r` instead of `Proxy` into the product yields `Product (Const r) Identity a ≅ (r, a) ≅ Writer r a`, representing the Minimal Monad of Logging.)

Chapter 7: Functors entirely out of Proxy

Section 7.1: The Fundamental Rule

Before combining multiple Proxies, there is a fundamental rule in the algebra of algebraic data types to remember:

Because `Proxy` holds exactly zero values of type `a`, `Proxy` perfectly represents the number 1 (a single state with no `a` data attached).

Anytime you want to introduce an "empty" case to a data structure (like `Nothing` in `Maybe`, or `[]` in `List`, or `Leaf` in a Tree that holds no data), you are fundamentally using `Proxy` (or `Const ()`) mixed into your sum type!

Section 7.2: Proxy + Proxy = Const Bool

What if we sum two Proxies together?

```
Sum Proxy Proxy a  ≅  Either (Proxy a) (Proxy a)  ≅  Either () ()
```

An `Either () ()` type has exactly two possible values (`Left ()` or `Right ()`). That is exactly a Boolean! So summing two Proxies creates the **`Const Bool`** functor. Mathematically: $1 + 1 = 2$

Section 7.3: Proxy * Proxy = Proxy

What if we take the product of two Proxies?

```
Product Proxy Proxy a  ≅  (Proxy a, Proxy a)  ≅  ((), ())  ≅  Proxy a
```

A tuple of two unit types `((), ())` still holds exactly one possible uninteresting value. Thus, it collapses back down to `Proxy`! Mathematically: $1 \times 1 = 1$

Chapter 8: Recursion and Fixed Points

Once we understand combinations of `Proxy` and `Identity`, we can create infinitely large structures via recursion.

Section 8.1: List (Recursive Sums & Products)

How do we build the standard `List` functor? A list is either empty (`[]`), or it has a head and a tail (`x : xs`). * The empty case holds zero `a`s. That is **`Proxy`**. * The head + tail case holds one `a` and the rest of the list. That is **`Product Identity List`**.

Therefore, the formula is: `List a ≅ Sum Proxy (Product Identity List) a`

Algebraically: $L(X) = 1 + X \times L(X)$

Section 8.2: Is $1 + X \times W = W$ always the case?

Looking at the list equation, you might ask: "is it always the case that `Sum Proxy (Product Identity Whatever) = Whatever`?"

The answer is no! The formula $1 + X \times W$ describes the "shape" of a single layer of a List. When we say $L(X) = 1 + X \times L(X)$, we are saying that `List` is exactly the type that satisfies this equation (it is the *Fixed Point* of that functor). If `Whatever` was a Binary Tree, its shape equation would look entirely different, such as $T(X) = 1 + X \times T(X) \times T(X)$.

Different mathematical formulas create different data structures!

Annex: Proofs and Derivations

Proof of Monad Equivalence

If `bind`, `join`, and `kleisli` are equivalent, we should be able to derive them algebraically from one another (assuming we have `fmap` and `pure`).

Deriving `join` from `bind`:

```
join mm = mm >>= id
-- Where mm :: m (m a) and id :: m a -> m a
```

Deriving `bind` from `join` and `fmap`:

```
m >>= f = join (fmap f m)
-- fmap f m produces m (m b)
-- join then flattens it to m b
```

Deriving `kleisli` from `bind`:

```
(f >=> g) x = f x >>= g
```

Proof of Unique Functor Identity

Given `data MinF a = Val`, how do we formally prove the only valid function of type `(a -> a) -> MinF a -> MinF a` preserving structure is the identity?

1. Let `g :: MinF a -> MinF a` be a total, terminating function.
2. The only inhabited value of `MinF a` at the term level is `Val`.
3. Therefore, `g Val = Val`.
4. By definition, the `id` function is `id x = x`.
5. Thus, `id Val = Val`.
6. Since `g Val = id Val` for the sole value of the type, `g = id`. Because `g` is the only total mapping, `fmap id = id` trivially holds, and no other lawful interpretation exists.

Bibliography

- **"Theorems for free!"** by Philip Wadler (1989).
- **"Notions of computation and monads"** by Eugenio Moggi (1991).
- **"Fast and Loose Reasoning is Morally Correct"** by Nils Anders Danielsson, John Hughes, Patrik Jansson, and Jeremy Gibbons (2006).
- **"The Typeclassopedia"** by Brent Yorgey (The Monad Reader Issue 13, 2009).
- *(Recommended Reading)* **"Thinking with Types"** by Sandy Maguire.
- *(Recommended Reading)* **"Functors, Applicatives, And Monads In Pictures"** by Aditya Bhargava.
- **"Category Theory for Programmers"** (Introductory Notes) by Bartosz Milewski (PDF Link).